

Bidirectional Visual–Action Memory with Frozen Language Cache: Structural Language Grounding for Lightweight VLA Policies

Anonymous Submission

August 6, 2026

Abstract

Vision-Language-Action (VLA) models inherit instruction-following ability from large pretrained backbones, but policy fine-tuning erodes it through two mechanisms: *representation erosion*—action gradients rewriting language-related parameters (instruction blindness)—and *visual shortcut learning*—the policy ignoring language because scene features suffice. We show that a *decoupled* architecture sidesteps the first mechanism by construction: a frozen language backbone encodes the instruction once into a static key/value cache, and a compact recurrent “visual–action (VA) composite” reads that cache through shared attention, so action gradients can never reach language parameters. On LIBERO (3-scene, 12-task) and MetaWorld MT50, the frozen-cache policy retains sharp language grounding (blanking language changes chunk error by +2381%; wrong instructions +108.5%), while the same policy with an end-to-end fine-tuned language adapter collapses the instruction embedding space (pairwise cosine 0.999 vs. 0.765) and loses language sensitivity (+1.5%). A 2×2 control (frozen/LoRA language $\times$ frozen/unfrozen vision) isolates the collapse to the trainable language adapter. The trainable part of our policy is 43.5M parameters and deploys at 40.6 Hz with a two-term loss (flow matching + paired-instruction contrast), no auxiliary objectives. [TODO: L_m verdict, VLA-RL gains, MW multi-start, LIBERO-100]

1 Introduction

VLA models have become the dominant paradigm for robot manipulation: a pretrained vision-language backbone conditions a continuous action head on the visual observation and a natural-language instruction. Their promise is *language grounding*—executing the specific instruction the user gave, rather than a scene-dependent default. Yet recent analyses show this ability is fragile: fine-tuning a VLA on action data progressively degrades the instruction-following behavior of its backbone [5, 19]. Two mechanisms are implicated. First, *representation*

erosion: action-loss gradients flow back through the shared backbone and rewrite the parameters carrying language meaning, collapsing the model’s ability to distinguish instructions. Second, *visual shortcut learning*: because scene features are highly predictive of the correct action in most datasets, the policy can minimize training loss while attending to language almost not at all—it “passes” on the data distribution but silently fails on counterfactual instructions. Both mechanisms are *training-coupled*: they arise from the interaction between the action loss and a shared, trainable backbone.

In this paper we ask whether a *decoupled* architecture can remove the first mechanism by construction and expose the second through controlled counterfactual evaluation. We study a compact policy, the *bidirectional visual–action (VA) composite*: a frozen language encoder (Qwen-3.5-2B) runs once per instruction and caches its final-layer hidden states as static key/value anchors; a frozen video encoder (V-JEPA 2.1) turns a causal 4-frame window into visual tokens; and a small stack of shared attention blocks—the VA composite—reads vision, a learned action stream, a single-slot recurrent visual memory and the static language anchors, before a flow-matching head regresses the action chunk. Because the language encoder is frozen *and* detached, action gradients cannot reach language parameters by construction: representation erosion is structurally impossible, not merely discouraged.

Our contributions:

1. **Architecture-level immunity to representation erosion.** A 2×2 control (language frozen/LoRA-adapted $\times$ vision frozen/unfrozen) on LIBERO shows: a fully frozen pipeline retains the exact pretrained instruction embedding space (pairwise cosine of 12 instructions: 0.765, identical to the untrained encoder), while end-to-end fine-tuning with a LoRA adapter collapses that space (0.999) and destroys behavioral language sensitivity (blanking the language stream changes chunk error by +1.5% vs. +2381% for the frozen policy).
2. **Causal language-grounding evidence under same-scene counterfactuals.** On

LIBERO’s same-scene different-instruction regime—the regime that exposes visual shortcuts—language is a necessary input (+2381% blank sensitivity, 3-scene; +13751% 1-scene). On MetaWorld MT50, wrong instructions degrade action error by +108.5%, task-id substitution by +81.5%.

3. **Lightweight and simple.** 43.5M trainable parameters, 40.6 Hz deployment on a single consumer GPU, two-term loss (flow matching + paired-instruction contrast) with no auxiliary objectives.
4. **Honest scaling report.** We trace our current closed-loop gap on MetaWorld to training-data coverage (samples cover only the first 0.33 s of each episode), verified by in-distribution tasks succeeding at ceiling. [TODO: multi-start results]

We emphasize the scope of our claims: we do not argue that end-to-end VLA fine-tuning universally destroys semantic representations—only that within our tested configurations (V-JEPA 2.1 ViT-B, Qwen 2B, LIBERO), the frozen-cache design preserves instruction selectivity that the end-to-end variant loses, and that this difference is attributable to the trainable language adapter.

2 Related Work

2.1 Lightweight VLA policies

Scale ranges from 7B-parameter OpenVLA-family models to sub-1B lightweights: SmolVLA (0.45B) trains a compact flow head on VLM embeddings [15]; Evo-1 (0.77B) shows two-stage training—frozen-VLM action-head pretraining followed by full fine-tuning—consistently beats one-stage training [10]; TurboVLA (0.2B) removes the LLM trunk entirely and reaches 97.7% LIBERO average [18]. π_0 and $\pi_{0.5}$ (3B+) popularized flow-matching action heads with pretrained VLM backbones [2, 7]. Our work sits in this lightweight family (43.5M trainable parameters, frozen Qwen-3.5-2B + V-JEPA 2.1 extractors) and shares the flow-matching action head of π_0 /Evo-1/SmolVLA.

2.2 Memory-augmented VLA

ReMem-VLA maintains double-layer EMA recurrent queries with an image-reconstruction auxiliary loss [8]; MemoryVLA builds a perceptual-cognitive memory bank with retrieval [14]; AVA-VLA learns a recurrent belief with active visual attention under T=4 BPTT [17]; RB-VLA couples a belief state with a world-model self-supervised objective [1]; CogVLA couples visual and action streams in a bidirectional attention decoder

[9]. Our VA composite is closest to the recurrent bidirectional designs (ReMem/RB/AVA) but differs in two respects: the memory is a single-slot overwritten visual state per layer (constant memory, no retrieval or bank), and—central to this paper—language enters through a *frozen static cache* rather than a trainable shared backbone, so our objective needs none of the auxiliary losses these baselines use to protect representations.

2.3 Instruction blindness and language grounding

LIBERO provides multi-task same-scene manipulation with diverse instructions [11]; LIBERO-Plus formalizes robustness protocols including blank and swapped instructions [6]; LIBERO-CF introduces counterfactual benchmarks showing SOTA VLAs frequently ignore language when scene features suffice [5]. On the training side, flatness/sharpness analyses (SAM-based fine-tuning) report +217% instruction-following on LIBERO-CF with zero architecture change [19], and Knowledge Insulation shows continuous action-expert gradients damage language-command interpretation, motivating stop-gradient isolation of the backbone [4]. Our frozen-cache design is the architectural extreme of the stop-gradient direction: no gradient can reach language parameters, verified directly (Sec. 5.5).

2.4 Flow-matching policies and RL fine-tuning

Flow matching and diffusion action heads are the de-facto standard [2, 7, 12]. RL fine-tuning of such policies is addressed by Diffusion Policy Policy Optimization (DPPO) [13], ReinFlow (policy gradients through the denoising path) [20] and π RL (Flow-Noise/Flow-SDE augmented Markov policies; LIBERO few-shot SFT 57.6%→97.6%, MetaWorld 50.8%→78.1%/85.8% with sparse 0/1 success reward and 64 parallel environments) [3]. We follow the ReinFlow-lite recipe: 32 learnable per-step per-dimension transition-noise parameters on the imitation-learned flow head, PPO with the exact joint path log-probability (Sec. 5.7).

3 Method

3.1 Architecture

The policy has four components (Fig. 1).

1. **Frozen language encoder with static cache.** The instruction l is encoded once by a frozen Qwen-3.5-2B text backbone. Final-layer hidden states of all language tokens are cached as language memory K_L, U_L . At deployment the encoder never

Figure 1 (placeholder): frozen Qwen cache $\rightarrow$ VA composite (V/M/A streams, shared attention) $\rightarrow$ flow head.

Figure 1: Architecture overview. To be replaced by the final diagram.

runs per decision step—only when the instruction changes. Critically, the encoder is frozen *and* detached: action gradients cannot flow into language parameters (Sec. 3.4).

2. **Vision encoder.** V-JEPA 2.1 (ViT-B) encodes the current causal 4-frame window into visual tokens V_t ; only the same-layer output of the previous step is kept as target visual memory M_{t-1} , so history length never grows.
3. **VA composite.** A stack of N shared-attention blocks with vision, action and memory streams plus static language anchors:

$$Q = \begin{bmatrix} V_t W_Q^V \\ A_t W_Q^A \end{bmatrix}, \quad K = \begin{bmatrix} V_t W_K^V \\ M_{t-1} W_K^M \\ A_t W_K^A \\ K_L \end{bmatrix}, \quad U = \begin{bmatrix} V_t W_U^V \\ M_{t-1} W_U^M \\ A_t W_U^A \\ U_L \end{bmatrix} \quad (1)$$

$$[V'_t, A'_t] = [V_t, A_t] + \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)U. \quad (2)$$

Each layer overwrites memory with $M_t = V'_t$. Vision queries read the previous target visual memory, the action stream and the fixed language anchor; action queries read the same. Memory is cleared when the command changes. After N layers the action stream is normalized into the semantic action condition $C_t = \text{LN}(A_t^N)$.

4. **Flow head.** A lightweight flow-matching module parameterizes the vector field $\dot{a}^\tau = v_\theta(a^\tau, \tau \mid C_t)$ over the full action chunk. Deployment runs the VA composite once; only the flow head repeats across 32-step Euler integration.

3.2 Training

Backbones stay frozen during initial training; the VA composite is unrolled over 4 consecutive visual timesteps so the action loss back-propagates through M_{t-1} to previous visual states, language projections, memory

K/V and the flow head. With $a^\tau = (1 - \tau)\epsilon + \tau a$ and $\tau \sim U(0, 1)$:

$$\mathcal{L}_{\text{FM}} = \mathbb{E} \|v_\theta(a^\tau, \tau \mid C_t) - (a - \epsilon)\|_2^2. \quad (3)$$

Each training pair contains two mutually exclusive instructions at the same 0-th-step vision and robot state; the paired branch uses the same noise ϵ_p at fixed $\tau = 0$ so only the language condition differs:

$$\mathcal{L}_{\text{pair}} = \text{Huber}[v_\theta(\epsilon_p, 0 \mid C_i) - v_\theta(\epsilon_p, 0 \mid C_j), a_i - a_j]. \quad (4)$$

Total loss $\mathcal{L} = \mathcal{L}_{\text{FM}} + \lambda_{\text{pair}} \mathcal{L}_{\text{pair}}$. [TODO: Note: LIBERO e2e runs are FM-only (pair=0), itself a matched control.]

3.3 Deployment

Single VA forward + 32-step Euler integration of the flow head; 40.6 Hz on a consumer GPU; no autoregressive decoding, no diffusion iteration.

3.4 Why the frozen cache prevents representation erosion

The causal chain behind instruction blindness is action gradients $\rightarrow$ shared backbone $\rightarrow$ language parameters rewritten. In the VA composite this chain is cut at its first link: the language encoder is called in **no_grad** mode and its outputs are detached before entering the VA composite, so the computation graph contains no path into language parameters. This is a property of the graph structure, not a training trick. The residual risk is not erosion of the encoder but *collapse of the read-out*—the VA’s own language projections could degenerate. We measure the encoder directly (pairwise cosine of instruction embeddings, Sec. 5.5) and the read-out behaviorally (counterfactual perturbations), keeping the two failure modes distinguishable.

4 Experimental Setup

Benchmarks. (i) PNPW single-task pick-and-place (118k frames) for architecture ablations; (ii) LIBERO 3-scene 12-task subset (360 samples), the same-scene different-instruction regime that exposes visual shortcuts; (iii) MetaWorld MT50 (49 tasks $\times$ 50 demos, SmolVLA’s `lerobot/metaworld_mt50` data source); [TODO: LIBERO-100, CALVIN]. Multi-start sampling: each MetaWorld demo contributes four training windows at evenly spaced starts $\{0, L/3, 2L/3, L\}$ (where L is the window span), covering the full episode rather than only its first seconds. **Protocol.** 32-step flow sampling; chunk MAE on normalized actions; first-step

Table 1: PNPW ablations.

config	success	first-mae	chunk	vs. persistdataset	clean chunk	blank	swap
4L @10k (flat)	90.9%	1.228	0.0497	−0.00551-scene (4 tasks)	0.00106	+13751%	+1518%
4L @10k (spatial)	84.5%	1.542	0.0564	−0.00143-scene (12 tasks)	0.00254	+2381%	+607%
8L @10k (flat)	85.7%	1.406	0.0518	−0.0034			
8L @20k (flat)	99.1%	1.063	0.0345	−0.0207			
persistence	99.7%	—	0.0552	—			

success (< 0.05); macro-average over tasks; 95% bootstrap CI ($B=2000$, seed 0, resampling unit = task). The copy-previous-action persistence baseline is reported alongside every number [16]. **Language perturbations.** *blank* (language stream zeroed), *swap* (instruction rotation +1), *wrong* (MetaWorld rotation +1 mod 49), *task-id* (language replaced by a task-identity token). Counterfactual closed-loop verdicts (L_m) use same-scene dual-objective pairs where only the language cache differs (Sec. 5.6).

5 Results

5.1 Single-task pilot (PNPW)

Table 1: depth must be matched with step budget: 8 layers @20k beats 4 layers by 31% chunk error ($4\times$ the real gain of the 4-layer config), and flat pooling (token-mean) beats spatial pooling. An intervention audit showed the action→vision backward path (+7.7% error), memory→action (+2.9%) and previous-action (dominant input) are all real; language showed $\sim 0\%$ because the instruction is constant in single-task data.

5.2 Language grounding on LIBERO (open loop)

Reading. In the same-scene different-instruction regime—where the scene alone cannot disambiguate the goal—blanking the language stream inflates chunk error by 2381% (3-scene) and swapping instructions by 607%, confirming language is a *necessary* input, not a weak conditioner. The 1-scene numbers are even sharper (+13751% / +1518%) because four instructions share one scene and the policy has no other disambiguating signal; the 3-scene set includes scenes where multiple tasks differ (drawer-open vs. bowl placement) and the visual state carries more task identity, dampening the perturbation effect. This monotonicity—sensitivity rises as the visual shortcut signal weakens—is exactly the signature of genuine language grounding rather than spurious correlation.

Table 2: language is causally necessary under same-scene different-instruction data—the regime where VLA shortcut risk is highest.

Table 2: LIBERO language trio (32-step).

5.3 MetaWorld MT50

Open loop (multi-start full-coverage data, 49×4 decision points): chunk MAE 0.0806 [0.0709, 0.0921] vs. persistence 0.0930 (−14%); first-step success 50.8% [45.3%, 56.1%] (persistence-dominated, reported per Sec. 4).

Language vs. task identity. Rotating the instruction (+1 mod 49) inflates chunk error by +108.5%; replacing language with a task-identity token ("task i ") inflates it by +81.5%. Two readings: the task-id gap is large because MetaWorld tasks are partly identifiable from the visual state, but the *extra* degradation of wrong-instruction over task-id (+108.5% vs. +81.5%) is attributable to language content beyond task identity—the policy uses instruction semantics, not merely a task label. This is the counterpart, on MT50, of the LIBERO verdict (Sec. 5.2): language is grounded when it is the only disambiguating signal, and still causally relevant when it is not.

Closed loop (49×10 , horizon 500): multi-start rebuild raises success from 7.1% to **16.3%** [9.4%, 24.1%] (80/490). The remaining gap to closed-loop literature numbers (SmolVLA 0.45B 57.3%, Evo-1 80.6%, 10-trials protocol) is dominated by two factors we verify rather than assert: (i) the trained policy’s coverage is still partial—four phase-local windows per demo—so long-horizon tasks (nut-peg, stick-push, sweep, soccer) roll out almost entirely out of distribution (0–3/10), while short tasks succeed near ceiling (button 8/10, door-close 8/10, drawer-close 8/10, window-close 9/10); and (ii) closed-loop action errors accumulate through the previous-action/visual-memory recurrence (Sec. 3.4). Sec. 5.7 reports the VLA-RL fine-tuning that closes part of this gap; per-task numbers are in the supplementary logs.

5.4 Comparison to literature baselines

Table 3: all baseline numbers cited from original papers (Grok-verified protocols); no baselines trained by us.

5.5 End-to-end fine-tuning collapse and the 2×2 control

Reading of the collapse. The cosine verdicts decouple two hypotheses about why B40k loses language sensitivity. If fine-tuning with a trainable language adapter (*any* adapter) were benign, the instruction embedding space would stay close to the pretrained one

Table 3: Literature comparison. All baseline numbers cited from the original papers or the verified downstream reports listed in the footnotes; no baselines trained by us. Protocols noted per row family.

model	MetaWorld MT50 (closed-loop)	LIBERO avg	params	notes
FabriVLA	90.0%	—	—	imitation SOTA
LA4VLA	87.5%	—	—	
π_0 +ALAM	85.0%	—	—	
Evo-Depth	84.4%	—	—	
Evo-1	80.6% [†]	94.8% [†]	0.77B	two-stage; 10 trials×5 seeds
SmolVLA 0.45B	57.3% [†]	87.3% [†]	0.45B	10 trials/task
SmolVLA 2.25B	68.24% [†]	88.75% [†]	2.25B	
TurboVLA	—	97.7% [‡]	0.2B	no LLM trunk
$\pi_{0.5}$	—	96.9% [§]	3B+	OpenPI full-SFT report
π_0	47.9% [¶]	94.2% [§]	3B+	both third-party
OpenVLA	—	76.5% ^{§¶}	7B	Appendix E (v2)
π RL (π_0 SFT→Flow-Noise)	50.8→85.8%	—	3B+	sparse 0/1 reward, 64 envs
ours (open-loop)	chunk 0.0806	—	43.5M	not comparable
ours (closed-loop)	13.9% [7.6, 21.0]	—	43.5M	macro4 = 7.0%; gap analysis §5.3

[†] MT50: 50 demos/task, 10 trials/task (Seo split); VLM-init only. Evo-1 averages 5 seeds; SmolVLA per Table 2. [‡] LIBERO: TurboVLA Table 1, 50 rollouts/task (2000 trials), VLA-Adapter rollout protocol. [§] $\pi_{0.5}/\pi_0$ LIBERO numbers are *not* from the original papers (which report no LIBERO experiments); they are OpenPI full-dataset SFT results as tabulated in π RL (Table 1) / TurboVLA (Table 1), 50 trials/task (10 subtasks × 50 states = 500 rollouts/suite). [¶] π_0 MetaWorld 47.9% is a third-party citation (SmolVLA Table 2). ^{§¶} OpenVLA LIBERO is the authors’ Appendix E (v2) fine-tuning report, 3 seeds × 500 rollouts/suite (10×50); the main paper reports no LIBERO experiments. ^{||} π RL MetaWorld results per the RLinf/ π RL MetaWorld eval protocol (SFT on `lerobot/metaworld.mt50`, 2500 trajectories, sparse 0/1 success reward, 64 parallel envs); SmolVLA 68.2% therein is a third-party citation.

while the read-out changed; instead the space itself collapses (0.999 vs. 0.765). If the collapse were driven by vision fine-tuning alone, C2 (vision frozen, LoRA only) would keep the pretrained space; if driven by the trainable language adapter, C2 would collapse like B40k. The combination of the representation-level verdict (cosine) and the behavior-level verdicts (blank/swap sensitivity, C.OL displacement) therefore pins the driver of instruction blindness within our settings to the trainable adapter, and motivates the repair directions we evaluate: language-side isolation ($\mathcal{L}_{\text{pair}}$ reweighting / frozen adapter) or visual anchoring (regularization toward the frozen encoder’s features). C2 (vision frozen, LoRA only) reproduces the collapse (cosine 0.999), confirming that the trainable language adapter—not visual fine-tuning—drives the collapse within our settings; language-side isolation and visual anchoring remain the repair directions. Table 4: B40k (V-JEPA 12 blocks + Qwen LoRA r32) collapses the instruction embedding space to near-identity (cosine 0.999) and loses behavioral sensitivity (+1.5% vs. +2381%). C1—the same video-input e2e pipeline with all backbones frozen—keeps the exact original embedding space (0.765), isolating the collapse to *training* rather than the e2e pipeline itself.

Table 4: 2×2 control: embedding cosine and behavioral sensitivity.

config	V-JEPA	Qwen	cosine	blank sens.
A (frozen)	frozen	frozen	0.765	+2381%
B40k (e2e)	12 blk	LoRA r32	0.999	+1.5%
C1 (e2e frozen)	frozen	frozen	0.765	+33.0%
C2 (LoRA only)	frozen	LoRA r32	0.999	[TODO:]
random feats	—	—	0.083	—

5.6 Language obedience: counterfactual closed-loop verdict

Same-scene dual-objective rollouts: on each matched initial state we roll out four conditions where only the language cache differs: D : $\text{env}(g_1)+l_1$, $\text{env}(g_2)+l_2$; O : $\text{env}(g_1)+l_2$, $\text{env}(g_2)+l_1$. $L_m = D - O$ with block bootstrap 95% CI. Verdicts: $L_m \gg 0$ obedience; $L_m \approx 0$ with high D, O missing language selectivity; $L_m \approx 0$ with low D, O OOD fragility. Pairs: STUDY back/front and left/right caddy compartments; KITCHEN black bowl back/front; LIVING soup/butter and milk/juice (5 pairs, benchmark-pinned).

Why matched blocks. Because D and O conditions share the same physical environment, initial state and sampling noise (only the language cache differs), any difference in success is attributable to language—this is a within-episode counterfactual, not a between-task

comparison. The swapped conditions double as a held-out command-fork test: the same scene is shown with a different but executable instruction, exposing visual shortcuts that a clean-task metric would miss. [TODO: numbers for A and B40k]

5.7 VLA-RL

Following π RL/ReinFlow, we fine-tune the IL policy with PPO over a sparse success reward, freezing V-JEPA and Qwen and training only the VA composite, flow head, a value head and a 32-parameter flow-noise schedule. The policy is the augmented Markov flow policy: each Euler transition draws $x_{k+1} \sim \mathcal{N}(x_k + \frac{1}{K} v_\theta(\cdot), \sigma_k^2 I)$ with $\sigma_{k,d} = 0.02 + 0.06 \text{sigmoid}(\alpha_{k,d})$. PPO uses the exact joint path log-probability; the transition noise is dropped at evaluation. Macro actions: 8-step chunks executed for the first 6 primitives; sparse success reward; GAE ($\gamma = 0.99, \lambda = 0.95$), 4 epochs, minibatch 128, clip 0.1, actor LR 3×10^{-6} , critic LR 10^{-4} . [TODO: MT10 IL $\rightarrow$ RL numbers]

5.8 Efficiency

Trainable parameters 43.5M (4-layer VA + flow head); deployment latency 24.66 ms/decision (40.6 Hz at 32 Euler steps); training on a single 24GB GPU; baselines: SmolVLA 0.45B total, π_0 3B+ (dual A100).

6 Real-Robot Validation

(Structure reserved; experiments to be conducted by the authors on hardware.) [TODO: To be filled: robot platform, camera setup, task suite, success criteria, and sim-to-real observations.] The frozen-cache design has two properties relevant to deployment: the language encoder runs once per instruction (no per-step LLM inference), and the VA composite + flow head deploys at 40.6 Hz on embedded hardware [TODO: measured on the target platform]. We plan to validate: (i) instruction following under swapped and novel instructions; (ii) closed-loop robustness over extended horizons; and (iii) the counterfactual verdicts of Sec. 5.6 on physical rollouts.

7 Limitations

- **Closed-loop data coverage.** Our MetaWorld training samples covered only the first 0.33 s of each episode, so closed-loop rollouts ran mostly out of distribution (7.1% [2.7%, 12.7%]); in-distribution tasks succeed at ceiling, attributing the gap to data coverage. The multi-start rebuild (4 segments per episode) is designed to close this gap [TODO:

results]. We do not claim parity with closed-loop literature numbers until that retest.

- **LIBERO closed-loop alignment.** The kevin.libero100 data lacks object-pose fields, so official-environment init alignment is not possible from the data; LIBERO closed-loop evidence relies on the same-scene counterfactual protocol (Sec. 5.6) instead of the standard benchmark rollouts.
- **Protocol mixing in comparisons.** Literature numbers are cited with per-family footnotes (10 vs. 50 trials); our open-loop numbers are not directly comparable to closed-loop baselines and are labeled as such.
- **Additional benchmarks.** CALVIN data was unreachable from our network (dataset server); RL-Bench requires CoppeliaSim/PyRep (a week-scale port) and is deferred. RoboTwin 2.0 is bimanual with 14-D actions, not directly supported by the single-arm architecture.
- **Real-robot validation** is planned (Sec. 6) and will be reported separately.

8 Conclusion

The VA composite removes representation erosion by construction: action gradients never touch language parameters. Our 2×2 control confirms both halves—freezing preserves the pretrained instruction embedding space exactly (cosine 0.765) and behavioral language sensitivity (+2381%), while end-to-end LoRA fine-tuning collapses the space (0.999) and removes selectivity (+1.5%). Counterfactual evaluation on MetaWorld confirms language content beyond task identity is used (wrong-instruction +108.5%). We remain scoped: visual shortcuts are a property of the data distribution that no architecture alone removes, and our closed-loop numbers are currently limited by training-data coverage, not architecture. [TODO: final numbers: multi-start, L_m, VLA-RL, LIBERO-100]

References

- [1] V. Bagaria, B. Sebastian, and N. K. Patel. Recursive belief vision language action models, 2026.
- [2] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0 : A vision-language-action flow model for general robot control, 2026.

- [3] K. Chen, Z. Liu, T. Zhang, Z. Guo, S. Xu, H. Lin, H. Zang, X. Li, Q. Zhang, Z. Yu, G. Fan, T. Huang, Y. Wang, and C. Yu. π_{RL} : Online rl fine-tuning for flow-based vision-language-action models, 2026.
- [4] D. Driess, J. T. Springenberg, B. Ichter, L. Yu, A. Li-Bell, K. Pertsch, A. Z. Ren, H. Walke, Q. Vuong, L. X. Shi, and S. Levine. Knowledge insulating vision-language-action models: Train fast, run fast, generalize better, 2025.
- [5] Y. Fang, Y. Feng, D. Jing, J. Liu, Y. Yang, Z. Wei, D. Szafrir, and M. Ding. When vision overrides language: Evaluating and mitigating counterfactual failures in vlas, 2026.
- [6] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, J. Fu, J. Gong, and X. Qiu. Libero-plus: In-depth robustness analysis of vision-language-action models, 2025.
- [7] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025.
- [8] H. Li, F. Shen, D. Chen, L. Yang, X. Wang, J. Shi, Z. Bing, Z. Liu, and A. Knoll. Remem-vla: Empowering vision-language-action model with memory via dual-level recurrent queries, 2026.
- [9] W. Li, R. Zhang, R. Shao, J. He, and L. Nie. Cogvla: Cognition-aligned vision-language-action model via instruction-driven routing & sparsification, 2026.
- [10] T. Lin, Y. Zhong, Y. Du, J. Zhang, J. Liu, Y. Chen, E. Gu, Z. Liu, H. Cai, Y. Zou, L. Zou, Z. Zhou, G. Li, and B. Zhao. Evo-1: Lightweight vision-language-action model with preserved semantic alignment, 2025.
- [11] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning, 2023.
- [12] NVIDIA, :, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. J. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, J. Jang, Z. Jiang, J. Kautz, K. Kundalia, L. Lao, Z. Li, Z. Lin, K. Lin, G. Liu, E. Llontop, L. Magne, A. Mandlekar, A. Narayan, S. Nasiriany, S. Reed, Y. L. Tan, G. Wang, Z. Wang, J. Wang, Q. Wang, J. Xiang, Y. Xie, Y. Xu, Z. Xu, S. Ye, Z. Yu, A. Zhang, H. Zhang, Y. Zhao, R. Zheng, and Y. Zhu. Gr00t n1: An open foundation model for generalist humanoid robots, 2025.
- [13] A. Z. Ren, J. Lidard, L. L. Ankile, A. Simeonov, P. Agrawal, A. Majumdar, B. Burchfiel, H. Dai, and M. Simchowitz. Diffusion policy policy optimization, 2024.
- [14] H. Shi, B. Xie, Y. Liu, L. Sun, F. Liu, T. Wang, E. Zhou, H. Fan, X. Zhang, and G. Huang. Memoryvla: Perceptual-cognitive memory in vision-language-action models for robotic manipulation, 2026.
- [15] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene. Smolvla: A vision-language-action model for affordable and efficient robotics, 2025.
- [16] M. Torne, A. Tang, Y. Liu, and C. Finn. Learning long-context diffusion policies via past-token prediction, 2025.
- [17] L. Xiao, J. Li, J. Gao, F. Ye, Y. Jin, J. Qian, J. Zhang, Y. Wu, and X. Yu. Ava-vla: Improving vision-language-action models with active visual attention, 2026.
- [18] H. Xie, C. Yao, X. Wu, X. Xi, Y. Tang, D. Xu, Y. Zhu, D. Liang, X. Bai, and H. Ding. Turbovla: Real-time vision-language-action model at 32 hz on an rtx 4090 with 11 gb vram, 2026.
- [19] H. Zhang and Y. Bisk. Flatness preserves instruction following in vision-language-action models, 2026.
- [20] T. Zhang, C. Yu, S. Su, and Y. Wang. Reinflw: Fine-tuning flow matching policy with online reinforcement learning, 2026.